LABORATORIO 09

Autenticación JWT y autorización por roles

INTEGRANTE
Jesus Castillo Sumire

CURSO
Desarrollo de Aplicaciones Web

SEMANA
09 FECHA
19 de octubre de 2026

Introducción y objetivo

El objetivo de este laboratorio es construir desde cero una API REST segura que registre usuarios, autentique sus credenciales con Spring Security y emita un token JWT. Con ese token, la aplicación identifica al usuario en cada petición y verifica si su rol le permite acceder a la ruta solicitada.

Cómo se trabajó y cómo se logró

Creé un proyecto Spring Boot independiente, configuré la conexión con MySQL de XAMPP y preparé las entidades, repositorios y servicios de seguridad. Después implementé el registro limitado al rol USER, el inicio de sesión, la generación y validación de JWT, el filtro por petición y las restricciones para USER, ADMIN y MODERATOR. Verifiqué la compilación, el arranque conectado a XAMPP y las respuestas HTTP de los escenarios del laboratorio.

Herramientas y conexión

Trabajé con Visual Studio Code, Java 17, Spring Boot 3.4.5, Spring Data JPA/Hibernate, Spring Security, Bean Validation, Maven, MySQL Connector/J, JJWT 0.11.5, XAMPP con MariaDB 10.4.32 en localhost:3306 y Postman. La aplicación escucha en el puerto 8080 y usa la base independiente security_db.

Alcance y seguridad

El esquema tiene las tablas roles, users y user_roles. BCrypt almacena los hashes de contraseña. La cadena de Spring Security es sin estado. El registro público siempre asigna USER y rechaza solicitudes que intenten pedir ADMIN o MODERATOR; las cuentas privilegiadas se crean localmente para las pruebas.

Resultado observado

La aplicación inició correctamente, Hibernate creó las tres tablas y el inicializador dejó los roles y las cuentas de prueba disponibles. Los escenarios de ruta pública, falta de token, token inválido, autorización de usuario y permiso de administrador produjeron los códigos esperados.

01 · Base de datos y modelo relacional

La base security_db fue creada en MySQL de XAMPP. Se consultó information_schema después del arranque y se confirmaron las tablas roles, users y user_roles. La tabla intermedia conserva las dos claves foráneas y permite la relación muchos a muchos que solicita la guía.

Tabla	Campos principales	Uso
users	id, username, email, password	Identidad de cuenta y hash BCrypt. Username y email son únicos.
roles	id, name	Catálogo USER, ADMIN y MODERATOR.
user_roles	user_id, role_id	Relación muchos a muchos; sus dos campos forman la clave primaria compuesta.



La tabla intermedia convierte la relación muchos a muchos entre usuarios y roles.

SQL para crear el esquema

```
CREATE DATABASE IF NOT EXISTS security_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

Configuración de conexión y JWT

```
spring.application.name=lab09-jwt-security
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/security_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=tr
ue&serverTimezone=UTC
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Clave de desarrollo para el laboratorio. En despliegues debe definirse JWT_SECRET_BASE64.
jwt.secret=${JWT_SECRET_BASE64:VGhpcy1pcy1hLWRldi1vbmx5LXNlY3JldC1rZXktZm9yLUxhYjA5LTIwMjY=}
jwt.expiration-ms=3600000

app.bootstrap-admin.username=${BOOTSTRAP_ADMIN_USERNAME:admin}
app.bootstrap-admin.email=${BOOTSTRAP_ADMIN_EMAIL:admin@test.com}
app.bootstrap-admin.password=${BOOTSTRAP_ADMIN_PASSWORD:Admin@S09}
app.bootstrap-user.username=${BOOTSTRAP_USER_USERNAME:user1}
app.bootstrap-user.email=${BOOTSTRAP_USER_EMAIL:user@test.com}
app.bootstrap-user.password=${BOOTSTRAP_USER_PASSWORD:User@S09}
app.bootstrap-moderator.username=${BOOTSTRAP_MODERATOR_USERNAME:moderator}
app.bootstrap-moderator.email=${BOOTSTRAP_MODERATOR_EMAIL:moderator@test.com}
app.bootstrap-moderator.password=${BOOTSTRAP_MODERATOR_PASSWORD:Mod@S09}
```

La conexión incluye createDatabaseIfNotExist=true. Esto crea el esquema al conectarse si aún no existe y si el usuario de MySQL tiene permiso de creación. Hibernate crea las tablas mapeadas. La clave JWT guardada en las propiedades es solo de desarrollo; se puede reemplazar mediante la variable JWT_SECRET_BASE64.

02 · Estructura y flujo de autenticación

Las responsabilidades están separadas. Los modelos describen usuarios y roles; los repositorios consultan MySQL; UserDetailsServiceImpl adapta los usuarios al mecanismo de Spring Security; AuthController recibe registro y login; JwtUtils firma y valida tokens; JwtAuthFilter procesa el encabezado Bearer antes de llegar al controlador.

Paso	Qué ocurre
1. Login	Postman envía correo y contraseña a POST /api/auth/signin.
2. Autenticación	AuthenticationManager busca la cuenta y BCrypt compara el hash guardado.
3. Emisión	JwtUtils crea un token firmado con el correo como subject y una expiración de una hora.
4. Petición privada	Postman envía Authorization: Bearer <token>. JwtAuthFilter valida la firma y la expiración.
5. Permiso	Spring Security carga las autoridades en SecurityContextHolder y compara el rol con la ruta.
6. Respuesta	La API responde 200 si permite la operación, 401 si falta autenticación y 403 si falta el rol.

Relaciones JPA

AppUser declara @ManyToMany y mapea user_roles con user_id y role_id. Role contiene el nombre único del perfil. UserRepository localiza el usuario por correo; RoleRepository encuentra el perfil por nombre. La fuente del modelo se conserva en docs/modelo-relacional.mmd.

Estructura del proyecto

src/main/java/com/tecsup/lab09security contiene config, controller, dto, jwt, model, repository y service. src/main/resources contiene application.properties. database contiene el script de creación. postman incluye la colección de solicitudes. docs contiene el modelo y el flujo Mermaid.

03 · Endpoints y control de acceso

Método y ruta	Acceso	Respuesta esperada
POST /api/auth/signup	Público; crea USER	201 creado; 400 si pide rol privilegiado; 409 duplicado; 400 datos inválidos
POST /api/auth/signin	Público	200 con accessToken y tokenType Bearer; 401 credenciales incorrectas
GET /api/test/all	Público	200
GET /api/test/user	ROLE_USER o ROLE_ADMIN	200 autorizado; 401 sin token; 403 rol insuficiente
GET /api/test/admin	ROLE_ADMIN	200 autorizado; 401 sin token; 403 rol insuficiente
GET /api/test/mod	ROLE_MODERATOR	200 autorizado; 401 sin token; 403 rol insuficiente

Ejemplo de inicio de sesión

POST http://localhost:8080/api/auth/signin

```
{
  "email": "admin@test.com",
  "password": "Admin@S09"
}
```

La respuesta contiene accessToken, tokenType, id, username, email y roles. Para llamar a una ruta privada, configura Authorization → Bearer Token en Postman y pega el valor de accessToken.

Cuentas locales para repetir las pruebas

Correo	Contraseña local	Rol
admin@test.com	Admin@S09	ROLE_ADMIN
user@test.com	User@S09	ROLE_USER
moderator@test.com	Mod@S09	ROLE_MODERATOR

Las cuentas de demostración se crean una sola vez. Si el correo ya existe, el inicializador conserva esa cuenta y su contraseña vigente. La consulta de evidencia confirmó que los hashes no se exponen en las respuestas.

04 · Pruebas de Postman y resultados

La colección postman/Lab09-JWT.postman_collection.json contiene el registro, los tres inicios de sesión y las solicitudes para repetir los escenarios. Las pruebas se ejecutaron localmente contra la aplicación conectada a security_db.

Caso	Solicitud	Resultado verificado
1 · Ruta pública	GET /api/test/all sin token	200 OK
2 · Acceso sin token	GET /api/test/user	401 Unauthorized
3 · Token modificado	GET /api/test/user con token alterado	401 Unauthorized
4 · Token válido USER	GET /api/test/user con token USER	200 OK
5 · Rol insuficiente	GET /api/test/admin con token USER	403 Forbidden
6 · ADMIN autorizado	GET /api/test/admin con token ADMIN	200 OK

Comprobaciones complementarias

Operación	Resultado verificado
Registro normal con datos válidos	201 Created; se asigna únicamente ROLE_USER.
Repetir correo existente	409 Conflict.
Solicitar ADMIN durante signup	400 Bad Request; no se concede el privilegio.
Login con contraseña incorrecta	401 Unauthorized.
Ingreso del moderador a /api/test/mod	200 OK.
Persistencia local	3 tablas, 3 roles y 3 cuentas de demostración en security_db.

Ejecución local

1. Iniciar MySQL en XAMPP. 2. Abrir la carpeta lab09-jwt-security en Visual Studio Code. 3. Revisar usuario y contraseña de MySQL en application.properties o en las variables DB_USERNAME y DB_PASSWORD. 4. Ejecutar .\mvnw.cmd spring-boot:run desde PowerShell. 5. Importar la colección de Postman y enviar primero los tres requests Login para guardar los tokens.

05 · Observaciones

Observación 1. Al principio revisé el nombre del esquema con cuidado, porque security_db debía ser una base nueva e independiente de securitydb, que pertenece al laboratorio anterior. La aplicación ahora apunta expresamente a security_db.

Observación 2. Al iniciar el backend, Hibernate creó las tablas de usuarios, roles y su tabla de unión. Revisé information_schema y confirmé las dos claves foráneas, por lo que la relación muchos a muchos quedó persistida en MySQL.

Observación 3. El registro público asigna el rol USER y rechaza si el cuerpo solicita ADMIN o MODERATOR. Esta restricción evita que una cuenta nueva se conceda permisos administrativos desde Postman.

Observación 4. Una petición sin token o con un token alterado devuelve 401. Cuando el token es válido pero el usuario no tiene el rol requerido, la respuesta es 403; los dos errores representan situaciones distintas.

Observación 5. Las cuentas locales se crean solo si no existe su correo. Si cambio la contraseña de configuración después de iniciar el sistema, el inicializador no reemplaza la contraseña guardada en la base.

06 · Conclusiones

Conclusión 1. Con este laboratorio entendí mejor que autenticación y autorización son pasos relacionados, pero distintos. Primero comprobé quién inicia sesión y después Spring Security revisó qué podía consultar según su rol.

Conclusión 2. Implementé JWT para que el servidor pueda validar cada petición protegida sin mantener una sesión tradicional. El token lleva una firma y una fecha de expiración, y debe enviarse en la cabecera Authorization.

Conclusión 3. Guardé las contraseñas con BCrypt y comprobé que el login autentica comparando el texto ingresado con el hash de la base de datos. Así no necesito guardar la contraseña original en MySQL.

Conclusión 4. La relación entre usuarios y roles quedó representada con users, roles y user_roles. Al separar esas tablas puedo asignar más de un rol a una cuenta sin repetir sus datos personales.

Conclusión 5. Las pruebas de Postman me permitieron revisar tanto los accesos permitidos como los rechazos. Ver los códigos 401 y 403 en casos diferentes me ayudó a comprobar que el filtro JWT y las reglas de roles están trabajando.

Cierre

El Laboratorio 09 quedó como un proyecto independiente y reproducible. La compilación finalizó correctamente, el backend se conectó a MySQL de XAMPP y las rutas principales respondieron de acuerdo con el token y el rol de cada cuenta.

Anexo · Código fuente implementado

A continuación incluyo los archivos de configuración y las clases Java del proyecto. El código fuente editable también está dentro de la carpeta lab09-jwt-security del repositorio. El marco de cada bloque no tiene relleno; algunos renglones largos se parten visualmente para que entren en la página.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.5</version>
        <relativePath/>
    </parent>
    <groupId>com.tecsup</groupId>
    <artifactId>lab09-jwt-security</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>lab09-jwt-security</name>
    <description>API REST con autenticación JWT y autorización por roles</description>
    <properties>
        <java.version>17</java.version>
        <jjwt.version>0.11.5</jjwt.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>${jjwt.version}</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>${jjwt.version}</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>${jjwt.version}</version>
            <scope>runtime</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

database/create_security_db.sql

```
CREATE DATABASE IF NOT EXISTS security_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

src/main/resources/application.properties

```
spring.application.name=lab09-jwt-security
server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/security_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&se
rverTimezone=UTC
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Clave de desarrollo para el laboratorio. En despliegues debe definirse JWT_SECRET_BASE64.
jwt.secret=${JWT_SECRET_BASE64:VGhpcy1pcy1hLWRldi1vbmx5LXNlY3JldC1rZXktZm9yLUxhYjA5LTIwMjY=}
jwt.expiration-ms=3600000

app.bootstrap-admin.username=${BOOTSTRAP_ADMIN_USERNAME:admin}
app.bootstrap-admin.email=${BOOTSTRAP_ADMIN_EMAIL:admin@test.com}
app.bootstrap-admin.password=${BOOTSTRAP_ADMIN_PASSWORD:Admin@S09}
app.bootstrap-user.username=${BOOTSTRAP_USER_USERNAME:user1}
app.bootstrap-user.email=${BOOTSTRAP_USER_EMAIL:user@test.com}
app.bootstrap-user.password=${BOOTSTRAP_USER_PASSWORD:User@S09}
app.bootstrap-moderator.username=${BOOTSTRAP_MODERATOR_USERNAME:moderator}
app.bootstrap-moderator.email=${BOOTSTRAP_MODERATOR_EMAIL:moderator@test.com}
app.bootstrap-moderator.password=${BOOTSTRAP_MODERATOR_PASSWORD:Mod@S09}
```

src/main/java/com/tecsup/lab09security/config/DataInitializer.java

```
package com.tecsup.lab09security.config;

import com.tecsup.lab09security.model.AppUser;
import com.tecsup.lab09security.model.Role;
import com.tecsup.lab09security.repository.RoleRepository;
import com.tecsup.lab09security.repository.UserRepository;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Configuration
public class DataInitializer {

    @Bean
    ApplicationRunner initializeSecurityData(RoleRepository roles, UserRepository users,
                                             PasswordEncoder encoder,
                                             @Value("${app.bootstrap-admin.username}") String username,
                                             @Value("${app.bootstrap-admin.email}") String email,
                                             @Value("${app.bootstrap-admin.password}") String password,
                                             @Value("${app.bootstrap-user.username}") String userName,
                                             @Value("${app.bootstrap-user.email}") String userEmail,
                                             @Value("${app.bootstrap-user.password}") String userPassword,
                                             @Value("${app.bootstrap-moderator.username}") String moderatorName,
                                             @Value("${app.bootstrap-moderator.email}") String moderatorEmail,
                                             @Value("${app.bootstrap-moderator.password}") String moderatorPassword) {
        return args -> initialize(roles, users, encoder,
                new String[][]{
                        {username, email, password, "ADMIN"},
                        {userName, userEmail, userPassword, "USER"},
                        {moderatorName, moderatorEmail, moderatorPassword, "MODERATOR"}
                });
    }

    @Transactional
    void initialize(RoleRepository roles, UserRepository users, PasswordEncoder encoder, String[][] accounts) {
        for (String roleName : List.of("USER", "ADMIN", "MODERATOR")) {
            if (roles.findByName(roleName).isEmpty()) {
                roles.save(new Role(roleName));
            }
        }
        for (String[] account : accounts) {
            String username = account[0];
            String email = account[1];
            String password = account[2];
            String roleName = account[3];
            if (!users.existsByEmailIgnoreCase(email)) {
                Role role = roles.findByName(roleName).orElseThrow();
                AppUser user = new AppUser(username, email, encoder.encode(password));
                user.addRole(role);
                users.save(user);
            }
        }
    }
}
```

src/main/java/com/tecsup/lab09security/config/SecurityConfig.java

```
package com.tecsup.lab09security.config;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.tecsup.lab09security.dto.MessageResponse;
import com.tecsup.lab09security.jwt.JwtAuthFilter;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.MediaType;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws Exception {
        return configuration.getAuthenticationManager();
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http, JwtAuthFilter jwtAuthFilter,
                                            ObjectMapper objectMapper) throws Exception {
        http.csrf(csrf -> csrf.disable())
                .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .exceptionHandling(exceptions -> exceptions
                        .authenticationEntryPoint((request, response, exception) -> writeError(
                                response, objectMapper, HttpServletResponse.SC_UNAUTHORIZED,
                                "Se requiere un token válido para acceder."))
                        .accessDeniedHandler((request, response, exception) -> writeError(
                                response, objectMapper, HttpServletResponse.SC_FORBIDDEN,
                                "El usuario autenticado no tiene permiso para esta ruta.")))
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/api/auth/**", "/api/test/all").permitAll()
                        .requestMatchers("/api/test/admin").hasRole("ADMIN")
                        .requestMatchers("/api/test/mod").hasRole("MODERATOR")
                        .requestMatchers("/api/test/user").hasAnyRole("USER", "ADMIN")
                        .anyRequest().authenticated())
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    private static void writeError(HttpServletResponse response, ObjectMapper mapper,
                                   int status, String message) throws java.io.IOException {
        response.setStatus(status);
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        mapper.writeValue(response.getOutputStream(), new MessageResponse(message));
    }
}
```

src/main/java/com/tecsup/lab09security/controller/AuthController.java

```
package com.tecsup.lab09security.controller;

import com.tecsup.lab09security.dto.*;
import com.tecsup.lab09security.jwt.JwtUtils;
import com.tecsup.lab09security.model.AppUser;
import com.tecsup.lab09security.model.Role;
import com.tecsup.lab09security.repository.RoleRepository;
import com.tecsup.lab09security.repository.UserRepository;
import com.tecsup.lab09security.service.UserDetailsImpl;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final UserRepository users;
    private final RoleRepository roles;
    private final PasswordEncoder passwordEncoder;
    private final AuthenticationManager authenticationManager;
    private final JwtUtils jwtUtils;

    public AuthController(UserRepository users, RoleRepository roles, PasswordEncoder passwordEncoder,
                          AuthenticationManager authenticationManager, JwtUtils jwtUtils) {
        this.users = users;
        this.roles = roles;
        this.passwordEncoder = passwordEncoder;
        this.authenticationManager = authenticationManager;
        this.jwtUtils = jwtUtils;
    }

    @PostMapping("/signup")
    @Transactional
    public ResponseEntity<MessageResponse> register(@Valid @RequestBody SignupRequest request) {
        if (request.roles() != null && request.roles().stream()
                .anyMatch(role -> !"USER".equalsIgnoreCase(role))) {
            return ResponseEntity.badRequest().body(new MessageResponse(
                    "El registro público solo permite el rol USER."));
        }
        if (users.existsByUsernameIgnoreCase(request.username().trim())) {
            return ResponseEntity.status(HttpStatus.CONFLICT)
                    .body(new MessageResponse("El nombre de usuario ya está registrado."));
        }
        if (users.existsByEmailIgnoreCase(request.email().trim())) {
            return ResponseEntity.status(HttpStatus.CONFLICT)
                    .body(new MessageResponse("El correo ya está registrado."));
        }
        Role userRole = roles.findByName("USER").orElseThrow();
        AppUser user = new AppUser(request.username().trim(), request.email().trim().toLowerCase(),
                passwordEncoder.encode(request.password()));
        user.addRole(userRole);
        users.save(user);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(new MessageResponse("Usuario registrado correctamente con rol USER."));
    }

    @PostMapping("/signin")
    public ResponseEntity<?> authenticate(@Valid @RequestBody LoginRequest request) {
        try {
            Authentication authentication = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(request.email(), request.password()));
            UserDetailsImpl principal = (UserDetailsImpl) authentication.getPrincipal();
            List<String> authorities = principal.getAuthorities().stream()
                    .map(authority -> authority.getAuthority()).toList();
            return ResponseEntity.ok(new JwtResponse(jwtUtils.generateJwtToken(authentication), "Bearer",
                    principal.getId(), principal.getDisplayName(), principal.getEmail(), authorities));
        } catch (AuthenticationException exception) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                    .body(new MessageResponse("Correo o contraseña incorrectos."));
        }
    }
}
```

src/main/java/com/tecsup/lab09security/controller/TestController.java

```
package com.tecsup.lab09security.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/test")
public class TestController {

    @GetMapping("/all")
    public Message all() { return new Message("Public Content"); }

    @GetMapping("/user")
    public Message user() { return new Message("User Content"); }

    @GetMapping("/admin")
    public Message admin() { return new Message("Admin Content"); }

    @GetMapping("/mod")
    public Message moderator() { return new Message("Moderator Content"); }

    public record Message(String message) { }
}
```

src/main/java/com/tecsup/lab09security/dto/JwtResponse.java

```
package com.tecsup.lab09security.dto;

import java.util.List;

public record JwtResponse(
        String accessToken,
        String tokenType,
        Long id,
        String username,
        String email,
        List<String> roles) {
}
```

src/main/java/com/tecsup/lab09security/dto/LoginRequest.java

```
package com.tecsup.lab09security.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public record LoginRequest(
        @NotBlank @Email String email,
        @NotBlank String password) {
}
```

src/main/java/com/tecsup/lab09security/dto/MessageResponse.java

```
package com.tecsup.lab09security.dto;

public record MessageResponse(String message) {
}
```

src/main/java/com/tecsup/lab09security/dto/SignupRequest.java

```
package com.tecsup.lab09security.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import java.util.Set;

public record SignupRequest(
        @NotBlank @Size(min = 3, max = 50) String username,
        @NotBlank @Email @Size(max = 120) String email,
        @NotBlank @Size(min = 8, max = 72) String password,
        Set<String> roles) {
}
```

src/main/java/com/tecsup/lab09security/jwt/JwtAuthFilter.java

```
package com.tecsup.lab09security.jwt;

import com.tecsup.lab09security.service.UserDetailsServiceImpl;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtUtils jwtUtils;
    private final UserDetailsServiceImpl userDetailsService;

    public JwtAuthFilter(JwtUtils jwtUtils, UserDetailsServiceImpl userDetailsService) {
        this.jwtUtils = jwtUtils;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        String header = request.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtUtils.validateJwtToken(token)
                    && SecurityContextHolder.getContext().getAuthentication() == null) {
                String email = jwtUtils.getEmailFromJwtToken(token);
                UserDetails userDetails = userDetailsService.loadUserByUsername(email);
                var authentication = new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

src/main/java/com/tecsup/lab09security/jwt/JwtUtils.java

```
package com.tecsup.lab09security.jwt;

import com.tecsup.lab09security.service.UserDetailsImpl;
import io.jsonwebtoken.*;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Component;
import java.security.Key;
import java.util.Date;

@Component
public class JwtUtils {

    @Value("${jwt.secret}")
    private String secretBase64;

    @Value("${jwt.expiration-ms}")
    private long expirationMs;

    private Key signingKey() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretBase64));
    }

    public String generateJwtToken(Authentication authentication) {
        UserDetailsImpl principal = (UserDetailsImpl) authentication.getPrincipal();
        Date issuedAt = new Date();
        return Jwts.builder()
                .setSubject(principal.getUsername())
                .setIssuedAt(issuedAt)
                .setExpiration(new Date(issuedAt.getTime() + expirationMs))
                .signWith(signingKey(), SignatureAlgorithm.HS256)
                .compact();
    }

    public String getEmailFromJwtToken(String token) {
        return Jwts.parserBuilder().setSigningKey(signingKey()).build()
                .parseClaimsJws(token).getBody().getSubject();
    }

    public boolean validateJwtToken(String token) {
        try {
            Jwts.parserBuilder().setSigningKey(signingKey()).build().parseClaimsJws(token);
            return true;
        } catch (JwtException | IllegalArgumentException exception) {
            return false;
        }
    }
}
```

src/main/java/com/tecsup/lab09security/Lab09SecurityApplication.java

```
package com.tecsup.lab09security;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Lab09SecurityApplication {

    public static void main(String[] args) {
        SpringApplication.run(Lab09SecurityApplication.class, args);
    }
}
```

src/main/java/com/tecsup/lab09security/model/AppUser.java

```
package com.tecsup.lab09security.model;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "users", uniqueConstraints = {
        @UniqueConstraint(name = "uk_users_username", columnNames = "username"),
        @UniqueConstraint(name = "uk_users_email", columnNames = "email")
})
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String username;

    @Column(nullable = false, length = 120)
    private String email;

    @Column(nullable = false, length = 100)
    private String password;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
            joinColumns = @JoinColumn(name = "user_id"),
            inverseJoinColumns = @JoinColumn(name = "role_id"),
            uniqueConstraints = @UniqueConstraint(name = "uk_user_role", columnNames = {"user_id", "role_id"}))
    private Set<Role> roles = new HashSet<>();

    protected AppUser() {
    }

    public AppUser(String username, String email, String password) {
        this.username = username;
        this.email = email;
        this.password = password;
    }

    public Long getId() {
        return id;
    }

    public String getUsername() {
        return username;
    }

    public String getEmail() {
        return email;
    }

    public String getPassword() {
        return password;
    }

    public Set<Role> getRoles() {
        return roles;
    }

    public void addRole(Role role) {
        roles.add(role);
    }
}
```

src/main/java/com/tecsup/lab09security/model/Role.java

```
package com.tecsup.lab09security.model;

import jakarta.persistence.*;

@Entity
@Table(name = "roles", uniqueConstraints = @UniqueConstraint(name = "uk_roles_name", columnNames = "name"))
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 30)
    private String name;

    protected Role() {
    }

    public Role(String name) {
        this.name = name;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

src/main/java/com/tecsup/lab09security/repository/RoleRepository.java

```
package com.tecsup.lab09security.repository;

import com.tecsup.lab09security.model.Role;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface RoleRepository extends JpaRepository<Role, Long> {
    Optional<Role> findByName(String name);
}
```

src/main/java/com/tecsup/lab09security/repository/UserRepository.java

```
package com.tecsup.lab09security.repository;

import com.tecsup.lab09security.model.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<AppUser, Long> {
    Optional<AppUser> findByEmailIgnoreCase(String email);
    boolean existsByUsernameIgnoreCase(String username);
    boolean existsByEmailIgnoreCase(String email);
}
```

src/main/java/com/tecsup/lab09security/service/UserDetailsImpl.java

```
package com.tecsup.lab09security.service;

import com.fasterxml.jackson.annotation.JsonIgnore;
import com.tecsup.lab09security.model.AppUser;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import java.util.Collection;

public class UserDetailsImpl implements UserDetails {

    private final Long id;
    private final String username;
    private final String email;
    @JsonIgnore
    private final String password;
    private final Collection<? extends GrantedAuthority> authorities;

    private UserDetailsImpl(AppUser user) {
        this.id = user.getId();
        this.username = user.getUsername();
        this.email = user.getEmail();
        this.password = user.getPassword();
        this.authorities = user.getRoles().stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role.getName()))
                .toList();
    }

    public static UserDetailsImpl build(AppUser user) {
        return new UserDetailsImpl(user);
    }

    public Long getId() { return id; }
    public String getDisplayName() { return username; }
    public String getEmail() { return email; }
    @Override public Collection<? extends GrantedAuthority> getAuthorities() { return authorities; }
    @Override public String getPassword() { return password; }
    @Override public String getUsername() { return email; }
    @Override public boolean isAccountNonExpired() { return true; }
    @Override public boolean isAccountNonLocked() { return true; }
    @Override public boolean isCredentialsNonExpired() { return true; }
    @Override public boolean isEnabled() { return true; }
}
```

src/main/java/com/tecsup/lab09security/service/UserDetailsServiceImpl.java

```
package com.tecsup.lab09security.service;

import com.tecsup.lab09security.repository.UserRepository;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class UserDetailsServiceImpl implements UserDetailsService {

    private final UserRepository userRepository;

    public UserDetailsServiceImpl(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        return userRepository.findByEmailIgnoreCase(email)
                .map(UserDetailsImpl::build)
                .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
    }
}
```